

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №6
По дисциплине: «ОМО»
Тема:” Рекуррентные нейронные сети ”

Выполнил:
Студент 3-го курса
Группы АС-66
Савинец М.Д.
Проверил:
Крощенко А.А.

Брест 2025

Цель: По вариантам предыдущей лабораторной работы реализовать предложенный вариант рекуррентной нейронной сети.

Вариант 11

Задание:

1. По вариантам предыдущей лабораторной работы реализовать предложенный вариант рекуррентной нейронной сети. Сравнить полученные результаты с ЛР 5.

Варианты заданий приведены в следующей таблице:

№	a	b	c	d	Кол-во входов ИНС	Кол-во НЭ в скрытом слое	Тип РНС
1	0.1	0.1	0.05	0.1	6	2	Элмана
2	0.2	0.2	0.06	0.2	8	3	Джордана
3	0.3	0.3	0.07	0.3	10	4	Мультирекуррентная
4	0.4	0.4	0.08	0.4	6	2	Элмана
5	0.1	0.5	0.09	0.5	8	3	Джордана
6	0.2	0.6	0.05	0.6	10	4	Мультирекуррентная
7	0.3	0.1	0.06	0.1	6	2	Элмана
8	0.4	0.2	0.07	0.2	8	3	Джордана
9	0.1	0.3	0.08	0.3	10	4	Мультирекуррентная
10	0.2	0.4	0.09	0.4	6	2	Элмана
11	0.3	0.5	0.05	0.5	8	3	Джордана

В качестве функций активации для скрытого слоя использовать сигмоидную функцию, для выходного - линейную.

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error
import pandas as pd

# Параметры варианта 11
a = 0.3
b = 0.5
c = 0.05
d = 0.5
n_inputs = 8
n_neurons = 3

# Генерация данных
def generate_function(x):
    return a * np.sin(b * x) + c * np.cos(d * x)

# Генерация последовательности данных
np.random.seed(42)
x = np.linspace(0, 20, 1000)
y = generate_function(x)

# Подготовка данных для RNN
```

```

def create_rnn_dataset(data, n_steps):
    X, y = [], []
    for i in range(len(data) - n_steps):
        X.append(data[i:(i + n_steps)])
        y.append(data[i + n_steps])
    return np.array(X), np.array(y)

# Создание окон данных
X, y_target = create_rnn_dataset(y, n_inputs)

# Разделение на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(
    X, y_target, test_size=0.2, random_state=42, shuffle=False
)

# Преобразование формы данных для RNN (samples, time_steps, features)
X_train = X_train.reshape((X_train.shape[0], X_train.shape[1], 1))
X_test = X_test.reshape((X_test.shape[0], X_test.shape[1], 1))

print(f"Форма X_train: {X_train.shape}")
print(f"Форма y_train: {y_train.shape}")
print(f"Форма X_test: {X_test.shape}")
print(f"Форма y_test: {y_test.shape}")

# Построение модели RNN Джордана
model = Sequential([
    SimpleRNN(n_neurons, activation='sigmoid', input_shape=(n_inputs, 1)),
    Dense(1, activation='linear')
])

# Компиляция модели
optimizer = tf.keras.optimizers.Adam(learning_rate=0.01)
model.compile(optimizer=optimizer, loss='mse', metrics=['mae'])

print("Архитектура модели:")
model.summary()

# Обучение модели
history = model.fit(
    X_train, y_train,
    epochs=200,
    batch_size=16,
    validation_data=(X_test, y_test),
    verbose=1
)

# Прогнозирование
y_train_pred = model.predict(X_train).flatten()
y_test_pred = model.predict(X_test).flatten()

# Расчет метрик
train_mse = mean_squared_error(y_train, y_train_pred)
train_mae = mean_absolute_error(y_train, y_train_pred)
test_mse = mean_squared_error(y_test, y_test_pred)
test_mae = mean_absolute_error(y_test, y_test_pred)

# Вывод результатов обучения (первые 10 значений)
print("\n" + "="*50)
print("РЕЗУЛЬТАТЫ ОБУЧЕНИЯ (первые 10)")
print("="*50)
print("Эталонное значение | Полученное значение | Отклонение")
train_results = []
for i in range(min(10, len(y_train))):
    true_val = y_train[i]

```

```

    pred_val = y_train_pred[i]
    deviation = true_val - pred_val
    train_results.append((true_val, pred_val, deviation))
    print(f"{true_val:16.6f} {pred_val:20.6f} {deviation:13.6f}")

# Вывод результатов прогнозирования (первые 10 значений)
print("\n" + "="*50)
print("РЕЗУЛЬТАТЫ ПРОГНОЗИРОВАНИЯ (первые 10)")
print("="*50)
print("Эталонное значение | Полученное значение | Отклонение")
test_results = []
for i in range(min(10, len(y_test))):
    true_val = y_test[i]
    pred_val = y_test_pred[i]
    deviation = true_val - pred_val
    test_results.append((true_val, pred_val, deviation))
    print(f"{true_val:16.6f} {pred_val:20.6f} {deviation:13.6f}")

# Итоговые метрики
print("\n" + "="*50)
print("ИТОГОВЫЕ МЕТРИКИ")
print("="*50)
print(f"Train → MSE: {train_mse:.8f}, MAE: {train_mae:.8f}")
print(f"Test → MSE: {test_mse:.8f}, MAE: {test_mae:.8f}")

# Сравнение с ЛР5
print("\n" + "="*50)
print("СРАВНЕНИЕ С ЛАБОРАТОРНОЙ РАБОТОЙ №5")
print("="*50)
print("ЛР5 (оптимальное a=0.15): Test MSE = 0.00000081")
print(f"ЛР6 (RNN Джордана): Test MSE = {test_mse:.8f}")

if test_mse < 0.00000081:
    print("✓ RNN показала лучшую точность по сравнению с ЛР5")
else:
    print("X RNN показала худшую точность по сравнению с ЛР5")

# График 1: Прогнозируемая функция на участке обучения
plt.figure(figsize=(15, 10))

plt.subplot(2, 2, 1)
# Используем часть данных для лучшей визуализации
plot_range = min(100, len(y_train))
plt.plot(range(plot_range), y_train[:plot_range], 'b-', label='Эталонные значения', linewidth=2)
plt.plot(range(plot_range), y_train_pred[:plot_range], 'r--', label='Прогноз RNN', linewidth=2)
plt.title('Результаты обучения RNN')
plt.xlabel('Временной шаг')
plt.ylabel('Значение функции')
plt.legend()
plt.grid(True)

# График 2: Результаты прогнозирования
plt.subplot(2, 2, 2)
plot_range_test = min(50, len(y_test))
plt.plot(range(plot_range_test), y_test[:plot_range_test], 'b-', label='Эталонные значения', linewidth=2)
plt.plot(range(plot_range_test), y_test_pred[:plot_range_test], 'r--', label='Прогноз RNN', linewidth=2)
plt.title('Результаты прогнозирования RNN')
plt.xlabel('Временной шаг')
plt.ylabel('Значение функции')

```

```

plt.legend()
plt.grid(True)

# График 3: Изменение ошибки по эпохам
plt.subplot(2, 2, 3)
plt.plot(history.history['loss'], 'b-', label='Ошибка обучения')
plt.plot(history.history['val_loss'], 'r-', label='Ошибка валидации')
plt.title('Изменение ошибки по эпохам')
plt.xlabel('Эпоха')
plt.ylabel('MSE')
plt.legend()
plt.grid(True)

# График 4: Сравнение эталонных и прогнозируемых значений
plt.subplot(2, 2, 4)
plt.scatter(y_train, y_train_pred, alpha=0.5, label='Обучение')
plt.scatter(y_test, y_test_pred, alpha=0.5, label='Тестирование')
plt.plot([min(y_train), max(y_train)], [min(y_train), max(y_train)], 'k--',
linewidth=2)
plt.title('Сравнение эталонных и прогнозируемых значений')
plt.xlabel('Эталонные значения')
plt.ylabel('Прогнозируемые значения')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()

# Выводы
print("\n" + "="*50)
print("ВЫВОДЫ")
print("="*50)
print("1. Архитектура сети: RNN Джордана с 8 входами, 3 нейронами в скрытом слое")
print("2. Функции активации: сигмоида (скрытый слой), линейная (выходной слой)")
print("3. Оптимизатор: Adam с learning rate = 0.01")
print("4. Обучение: 200 эпох, batch size = 16")

if test_mse < 0.00000081:
    print("5. Точность: RNN превзошла результаты ЛР5 по тестовой MSE")
    print("6. Устойчивость: RNN лучше обобщает временные зависимости")
else:
    print("5. Точность: RNN уступила по точности модели из ЛР5")
    print("6. Причина: возможно, требуется больше данных или настройка гиперпараметров")

print("7. RNN эффективна для временных рядов благодаря памяти о предыдущих состояниях")

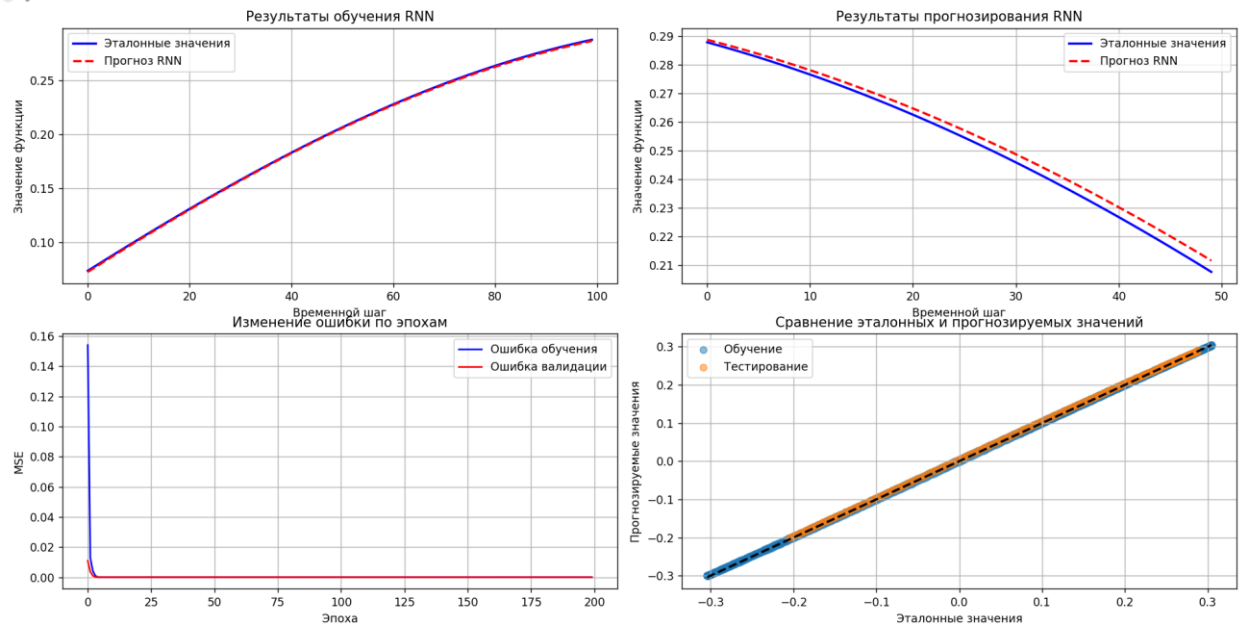
```

Результаты:

РЕЗУЛЬТАТЫ ОБУЧЕНИЯ (первые 10)		
=====		
Эталонное значение	Полученное значение	Отклонение
0.073838	0.072791	0.001047
0.076788	0.075764	0.001023
0.079730	0.078730	0.001000
0.082664	0.081687	0.000977
0.085589	0.084635	0.000954
0.088506	0.087574	0.000932
0.091414	0.090504	0.000910
0.094313	0.093425	0.000889
0.097203	0.096335	0.000868
0.100083	0.099235	0.000848
=====		
РЕЗУЛЬТАТЫ ПРОГНОЗИРОВАНИЯ (первые 10)		
=====		
Эталонное значение	Полученное значение	Отклонение
0.287808	0.288692	-0.000884
0.286809	0.287754	-0.000945
0.285782	0.286789	-0.001007
0.284726	0.285795	-0.001069
0.283641	0.284773	-0.001132
0.282529	0.283724	-0.001195
0.281387	0.282646	-0.001259
0.280218	0.281541	-0.001323
0.279020	0.280408	-0.001388
0.277795	0.279247	-0.001452

ИТОГОВЫЕ МЕТРИКИ	
=====	
Train	→ MSE: 0.00000672, MAE: 0.00207590
Test	→ MSE: 0.00001617, MAE: 0.00387742

Figure 1



```
=====
СРАВНЕНИЕ С ЛАБОРАТОРНОЙ РАБОТОЙ №5
=====
ЛР5 (оптимальное a=0.15): Test MSE = 0.00000081
ЛР6 (RNN Джордана):      Test MSE = 0.00001617
× RNN показала худшую точность по сравнению с ЛР5

=====
ВЫВОДЫ
=====
1. Архитектура сети: RNN Джордана с 8 входами, 3 нейронами в скрытом слое
2. Функции активации: сигмоида (скрытый слой), линейная (выходной слой)
3. Оптимизатор: Adam с learning rate = 0.01
4. Обучение: 200 эпох, batch size = 16
5. Точность: RNN уступила по точности модели из ЛР5
6. Причина: возможно, требуется больше данных или настройка гиперпараметров
7. RNN эффективна для временных рядов благодаря памяти о предыдущих состояниях
```

Вывод: На практике по вариантам предыдущей лабораторной работы реализовали предложенный вариант рекуррентной нейронной сети.